PROJECT HANDBOOK / VERSION 1.0

A local security assessment workflow

How the Northstar Supply demo, assessment engine, evidence store, and reports fit together. Written for project teammates, reviewers, and faculty.

Purpose: demonstrate a repeatable, evidence-backed security assessment on an intentionally vulnerable application that is restricted to a local loopback target.

At a glance

This project pairs a practical online store with a Python assessment framework. A person can use normal store functions, observe deliberately weak behavior, run an assessment from a dashboard, inspect how evidence becomes candidate findings, and review suggested fixes and retest evidence. It turns security concepts into a workflow that can be demonstrated and discussed end to end.

Question	Project answer
Problem	Security checks are often shown as isolated payloads or static screenshots. This project demonstrates how normal application routes, role boundaries, evidence, risk, reports, and retesting connect.
Audience	Students, faculty, teammates, and reviewers learning web application security assessment design.
Target	Northstar Supply, a local Flask and SQLite online store with intentional weaknesses and secure comparison routes.
Controller	Python modules for reconnaissance, role-aware checks, controlled probes, ZAP artifact import, evidence triage, risk scoring, attack paths, correlation, and reports.
Output	An assessment history, workflow-stage records, candidate findings, risk summaries, attack-path candidates, HTML and JSON reports, plus retest evidence.

Safety boundary: the controller accepts only plain HTTP to localhost or 127.0.0.1 on the configured port (currently 5001). Keep the vulnerable target local. Do not expose it to a campus or public network.

System map

The two web interfaces have different jobs. The storefront is the system under assessment. The assessment desk controls a run and presents its stored results. The controller does not depend on a browser to perform its checks; it makes bounded HTTP requests from the assessment code.

Layer	What it does	Key implementation
Northstar target	Normal shop flows and intentionally weak endpoints; data is stored in SQLite.	target-app/app.py, templates, static assets, Docker Compose
Assessment desk	Starts a run, displays history and stages, links to reports, records retest requests.	framework/dashboard.py and dashboard template
Assessment engine	Discovers endpoints by role, executes checks, persists observations, calculates risk, builds paths and correlations.	framework/main.py and recon/scanners/verification/risk modules
Persistence	Separate assessment database for targets, endpoints, identities, findings, evidence, paths, and stages.	framework/database/
Artifacts	Human-readable HTML and machine-readable JSON reports, correlation JSON, optional ZAP import/live-check JSON.	reports/

The data path

Run request -> local-target validation -> assessment and target records -> endpoint discovery per role -> authorization, ownership, and controlled vulnerability observations -> optional scanner import and safe live response checks -> evidence triage -> risk, paths, and correlation -> report generation -> dashboard and downloadable artifacts.

What lives where

- **target-app/** - Flask target, Docker Compose file, templates, design tokens, storefront styles, and local fixture files.
- **framework/** - assessment orchestration, the dashboard, database models and migrations, reconnaissance, probes, verification, risk, attack graph, and reporting.
- **reports/** - assessment-specific HTML/JSON, correlation artifacts, and ZAP-related JSON.
- **README.md** - canonical start, run, report, and retest commands.

Important separation

Northstar uses its own SQLite database for users, products, orders, and reviews. The framework uses a separate assessment database for its run records and evidence. The target container persists its database under target-app/instance. Assessment outputs are written under reports/. This separation lets the target data and assessment history be explained independently.

The assessment, stage by stage

A run is created before work begins. Each stage stores a status and a short detail string. A stage can succeed, fail, or be skipped. The overall run is COMPLETED only when the configured workflow and report generation succeed; otherwise it is PARTIAL.

Stage	What happens	What to explain
1. reconnaissance_and_role_mapping	Crawls the app as anonymous, user, and admin; probes configured API candidate paths; persists endpoint and access observations.	Discovery is scoped and role-aware. It records what responded, not whether every route is safe.
2. role_aware_authorization_and_idor	Compares observed access with configured role policies and compares authenticated identity with returned object owner.	Expected access comes from the project policy. An unexpected successful response is a candidate for review.
3. common_web_vulnerability_probes	Runs controlled SQL, reflected/stored/DOM XSS, CSRF-surface, path traversal, and security-configuration checks.	Probes are bounded to the local app. XSS checks inspect encoding or source/sink patterns; they do not execute scripts.
4. zap_candidate_import (optional input)	Imports an existing scope-matched ZAP JSON artifact if available.	The workflow does not launch ZAP. The checked-in artifact can be old; provide a fresh scoped report for current results.
5. live_response_verification (when ZAP input exists)	Makes safe live HTTP response checks for relevant imported alerts and stores the result JSON.	A response check is evidence, not automatic proof of exploitability.
6. verification_evidence_triage	Matches live check results to persisted findings and appends evidence.	Triage does not silently change finding lifecycle status.
7. risk_assessment	Calculates a project-specific score from impact, exploitability, and confidence.	The score is not CVSS and should not be presented as a standards-based severity score.
8. attack_path_analysis	Builds candidate paths from findings and their evidence; also looks for compound authorization exposure.	A path is an analysis hypothesis; report data explicitly avoids claiming confirmed exploitability.
9. finding_correlation	Correlates related observations, evidence, and endpoints.	Correlation helps reviewers prioritize relationships; it is not a new verification method.
10. report_generation	Writes the assessment-specific HTML and JSON report and refreshes final status.	The reports preserve stage status, evidence, limitations, risk, findings, and attack paths.

Example in the workspace: assessment #15 completed 10 of 10 stages. Its report contains 21 candidate findings, 7 candidate paths, and 21 endpoint observations. All 21 findings remain CANDIDATE; zero are marked verified or remediated.

Target behaviors and secure comparisons

The target is a realistic but small online shop, not a collection of static vulnerability cards. Its normal routes make the weakness observable. Paired secure routes show the relevant defensive pattern.

Area	Demonstration route	Comparison / defensive pattern
SQL injection	/api/products/search - query text is interpolated into SQL and database errors can be returned.	/api/products/search/secure - parameter binding.
Reflected XSS	/products/preview - request value is placed into HTML without escaping.	/products/preview/secure - HTML escaping.
Stored XSS	/reviews - stored visitor text is output as markup.	/reviews/secure - output escaping; assessment probe removes its harmless temporary record.
DOM XSS	/products/dom-preview - URL fragment reaches innerHTML.	/products/dom-preview/secure - textContent.
CSRF	/account/preferences - authenticated state-changing form has no token.	/account/preferences/secure - per-session token validated with constant-time comparison.
Path traversal	/api/files/download - name resolves within the lab fixture root, which includes the private fixture.	/api/files/download/secure - restricts resolution to the public fixture directory.
Access control	/admin/lab/users - deliberately unrestricted read-only user listing.	Admin APIs check login and role.
IDOR/BOLA	/api/orders/insecure/<id> - omits order ownership validation.	/api/orders/<id> checks ownership or admin role.
Headers / cookies	The target intentionally lacks some recommended hardening.	The assessment reports observed response conditions as candidates for review.

Store features

The storefront has a database-backed featured product and catalog, search, a session-based account sign-in, local checkout that creates sample orders without payment processing, order receipts, delivery preference flows, product reviews, offer preview, document retrieval, and admin operations. The Hallmark Editorial redesign keeps these application routes and handlers while using live product data, a non-fabricated empty review state, a restrained paper/ink/green palette, and clearer hierarchy.

Evidence, findings, and reports

The framework separates observations from conclusions. Reconnaissance records endpoints by role. Verification modules save expected-versus-observed responses. Vulnerability probes attach request and response details to evidence. The report presents these records alongside the lifecycle status so a reviewer can decide what to confirm.

Term	Meaning in this project
Observation	A response, endpoint, or behavior collected by a crawl or check.
Evidence	Structured data attached to a finding, with source, check name, target route, and observed result.
Candidate	An issue supported by an observation that still needs human review. This is the status of the example assessment findings.
Verified	A reviewer-supported status. Automated candidates do not become verified automatically.
Retest evidence	A later check appended to an existing finding after a code change. Retesting does not automatically change lifecycle status.
Candidate attack path	A structured relationship among one or more findings. It is not proof that an end-to-end exploit works.

Report files

For assessment ID N, the normal artifacts are reports/final-report-assessment-N.html and .json, reports/correlation-report-assessment-N.json, and, if ZAP verification ran, reports/zap/zap-live-verification-assessment-N.json. The desk exposes the HTML and JSON report with Open HTML report and Open JSON report links. Reports contain endpoint/role observations, stage records, evidence, risks, correlations, attack paths, and limitations.

Risk score

The project score uses three 1-to-5 inputs: impact weighted 40%, exploitability weighted 35%, and confidence weighted 25%, each normalized to 0-100. Severity labels are project thresholds. State plainly in a presentation that this is a preliminary project method, not CVSS.

Remediation and retesting

Fix the target code, restart the target, select the original assessment, and run Record remediation retest. The retest checks role access and order ownership against the assessment target, appends evidence, and refreshes the report. It intentionally leaves finding status and risk fields unchanged for human review. A passing check is evidence of improvement on the tested cases, not a universal proof that a vulnerability is gone.

Safety, scope, and limitations

- The controller rejects targets except HTTP localhost/127.0.0.1, the configured port, and the root path.
- The Docker port binds to 127.0.0.1:5001. Do not change it to a public interface for a class demo.
- Search and XSS probes are controlled. Stored-XSS comparison uses a harmless formatting marker and protected cleanup; no script executes.

- Path traversal checks stay within target-app/lab_files. The fixture is deliberately small and synthetic.
- CSRF assessment checks form/token and cookie conditions; it does not submit a victim action.
- The framework imports a supplied ZAP artifact; it does not start ZAP. Use a fresh artifact with correct scope for a current ZAP pass.
- The demo credentials and fixed secret are intentionally weak lab values. Never reuse them outside this local target.
- This is a teaching prototype, not a production scanner, penetration test, or compliance certification.

Teammate orientation

Task	Start here
Understand the app	README.md, then target-app/app.py and target-app/templates/store.html. Compare each vulnerable route with its secure counterpart.
Understand a run	framework/main.py; then follow the stage modules listed above.
Understand what evidence means	framework/verification/results.py, framework/verification/persist_results.py, and framework/verification/finding_manager.py.
Understand scoring and paths	framework/risk/scoring.py and framework/attack_graph/engine.py.
Understand output	framework/reporting/generate_report.py and reports/final-report-assessment-15.json.
Run locally	Use the two-terminal instructions in the companion demo runbook.

Suggested team review questions

- Can you trace one candidate from route -> observed behavior -> evidence record -> report row?
- What additional evidence would you require before changing a candidate to verified?
- Which secure comparison actually addresses the identified weakness, and what does it not cover?
- How would you make an imported scanner artifact fresh, scoped, and traceable to this run?

Detailed project brief and the problem we set out to solve

The starting presentation problem was that a training screen can look like a security project while showing fixed cards, canned vulnerability examples, and interactions that do not exercise an application. A viewer can read that a weakness exists, but cannot connect it to an ordinary business action, a real request, an observed response, an assessment record, and a remediation decision. That gap makes the demonstration feel hard-coded and makes it difficult for teammates to explain what the system actually does.

The product goal became a practical, locally runnable online shop with intentionally weak and paired secure behaviors. The assessment framework should interrogate the same routes that power shopping, account, review, receipt, and store operations flows. A single assessment action should create a durable run, execute focused modules, retain evidence, calculate preliminary risk, identify related candidate paths, generate reports, and allow later retesting.

Before / gap	What this implementation connects
Static lesson content	Store pages render live product data and use real Flask routes, form submissions, API requests, authentication sessions, SQLite records, and local files.
Vulnerability concepts detached from behavior	Weaknesses are represented by ordinary app endpoints, with separate secure comparison endpoints for teaching and review.
Manual screenshots without provenance	The assessment ID, target, stages, timestamps, endpoint/role observations, evidence, findings, paths, and report are persisted.
One scanner result treated as a verdict	Signals enter as candidates with source-specific evidence. Lifecycle transitions require review and, for remediation states, persisted retest evidence.
No connection between scanner and developer action	Finding rows carry remediation suggestions; the retest checks selected authorization and ownership cases after a fix.
One report format or ephemeral output	The dashboard serves HTML for people and JSON for structured review; the database keeps history and the report includes evidence context.
Unbounded demo target	The controller validates loopback host, plain HTTP, configured port, and root path; Docker binds the target to loopback.

Bridges built by the project

- **Business action -> endpoint:** search, checkout, account, review, receipt, preferences, and document lookup route to real handlers.
- **Endpoint -> security check:** the framework sends requests to configured routes and compares their live response or source pattern.
- **Identity -> policy:** anonymous, ordinary user, and admin observations are compared with a configured access matrix.
- **Behavior -> evidence:** each observation records the route, role, status, response metadata, expected result, and relevant comparison.

- **Evidence -> finding:** findings are deduplicated by assessment, title, and category; equivalent evidence is also deduplicated.
- **Finding -> prioritization:** known vulnerability categories receive a transparent project score with factors and method stored alongside it.
- **Findings -> path hypothesis:** supported evidence types become candidate paths, with linked finding IDs and evidence IDs.
- **Run -> reviewer artifact:** one ID ties database records to HTML/JSON reports and optional ZAP artifacts.
- **Fix -> new evidence:** retest appends timestamped checks while preserving the old result and human-controlled status.

> **Important interaction nuance: normal store actions do not stream click events into the assessment desk. The desk reflects behavior because a fresh assessment replays controlled checks against the same application routes and records the responses. The workflow is endpoint-linked and repeatable, not passive browser telemetry.**

What better means here

Compared with disconnected hard-coded examples, the implementation makes the demo reproducible and traceable. Product labels and prices come from the target database; a review is persisted; orders belong to users; a route has a response; the assessment stores what it observed; and reports link back to the assessment ID. The secure pair makes the defensive coding change visible. The design still requires human judgment: automation identifies evidence to inspect, not an authoritative security conclusion.

Northstar feature inventory: every demonstration flow

Feature	User action and client behavior	Server route and persistence	Assessment relationship
Catalog	Loads product list on page load; updates count; empty results have a state; featured item is rendered separately.	GET /api/products reads Product rows; / renders first Product as featured_product.	The crawler sees storefront/API paths. SQL checks use the search endpoint.
Search	Search form sends the query asynchronously; status text reports result count or failure; a shareable preview link appears for a non-empty query.	GET /api/products/search?q=... is deliberately vulnerable; /secure is paired parameterized route.	Scanner sends control, boolean, and error probes; preserves status/result counts and error disclosure.
Bag and checkout	Add-to-bag uses an in-memory list, updates button label and bag status, allows up to the server cap, and supports multiple products in one checkout request.	POST /cart/checkout requires login; creates one Order per product ID; returns order IDs/products/amounts; no payment processor.	Assessment observes auth and store routes; ownership checks use order records created by demo data or checkout.
Authentication	Login form posts username/password; successful login establishes Flask-Login session and redirects to account; logout requires login.	POST /login uses password hash verification; seeded lab users are alice, bob, and admin.	Framework logs in as configured user and admin; it does not use Bob in the central DEMO_ROLES map.
Account and orders	Account region reports signed-in state, exposes preferences, invoice, sign-out, and order list.	GET /orders returns current user's orders; GET /profile returns identity; dashboard route requires authentication.	User context is compared to object owner during IDOR observation and retest.
Receipts	Order rows link to a receipt lookup route. Store support has a numeric receipt lookup.	/api/orders/insecure/ is intentionally missing the owner check; /api/orders/ checks owner or admin.	IDOR verifier compares an owned object with a foreign object discovered through admin fixtures.
Reviews	Visitor posts up to 500 characters. The page refreshes the review iframe and offers a separate secure list link.	POST /reviews stores Review.body; GET /reviews inserts body unescaped. /reviews/secure escapes it. Admin list/delete routes are role-gated.	Probe writes a unique inert formatting marker, compares both views, then attempts protected cleanup.
Member offer preview	Offer form URL-encodes user term and opens the preview route in a new tab.	GET /products/preview reflects input unsafely; secure sibling escapes it.	Assessment compares reflected output bytes and the secure response; no script executes.
DOM preview	A direct page consumes the URL fragment as a preview.	Weak route assigns fragment to innerHTML; secure sibling assigns textContent.	Scanner checks static source/sink proximity in response source, not browser execution.
Delivery preferences	Preference form sends display_name; primary form has no token. A secure demonstration route issues and checks a per-session token.	POST /account/preferences writes a session value. /secure uses a random token and compare_digest.	Scanner authenticates, GETs both forms, checks token fields, and does not submit either form.

Feature	User action and client behavior	Server route and persistence	Assessment relationship
Document lookup	Store support submits a filename and displays returned content/error.	Vulnerable route resolves under lab_files, including its private subfolder; secure route constrains to public_files.	Probe reads the synthetic private marker inside the fixture tree, never host files.
Store operations	Admin screen displays user/order/report data and offers review moderation controls.	Admin API routes require login and role; one lab user-list route is intentionally open/read-only.	Policy-based role checks compare anonymous, Alice, and admin for configured endpoints.

Client behavior details

The redesigned store uses fetch for catalog load, checkout, order list, review submit, receipt lookup, and document lookup. DOM updates use textContent for those status and product fields. The vulnerable sinks are separate demonstration endpoints so the ordinary storefront does not need to execute a malicious script to teach the issue. Store styling and the Hallmark Editorial system are in target-app/static/tokens.css, base.css, and store.css; the assessment desk imports the same token and base layer through framework/static/dashboard.css.

Target route catalog and paired weakness map

Method + route	Authentication / role	Purpose and implementation note
GET /	Public	Store HTML, first Product featured from DB.
GET /api/products	Public	JSON product catalog.
GET /api/products/search	Public	SQL string interpolation, controlled error response; candidate sink.
GET /api/products/search/secure	Public	Uses bound SQL parameter.
GET /products/preview	Public	Reflects query into HTML without escaping.
GET /products/preview/secure	Public	HTML-escapes query output.
POST, GET /reviews	Public	Stores review and later emits raw markup in vulnerable listing.
POST /reviews/add/secure, GET /reviews/secure	Public	Secure write/list comparison; list escapes output.
GET /products/dom-preview	Public	Reads location.hash then writes innerHTML.
GET /products/dom-preview/secure	Public	Uses textContent sink.
POST /login, GET /logout	Public / authenticated	Hash-based password check and session lifecycle.
GET /dashboard, /profile, /orders	Authenticated user	Account page, identity JSON, only current user's orders.
GET /api/orders/<id>	Authenticated user/admin	Returns 403 for another user's order; admin may read.
GET /api/orders/insecure/<id>	Authenticated user	No ownership check; IDOR/BOLA demonstration.
GET /admin/users, /admin/orders, /api/admin/reports	Admin only	Protected user/order/report APIs.
GET /admin/lab/users	No auth check	Intentional unrestricted read-only user list for authorization testing.
GET /admin	Authenticated	Operations interface; display API results as text.
GET /admin/reviews, DELETE /admin/reviews/<id>	Admin only	List and clean reviews. Supports demonstration moderation and scanner marker cleanup.
POST /cart/checkout	Authenticated	Creates sample orders from selected Product IDs; limit 20 items.
GET, POST /account/preferences	Authenticated	Session preference mutation without CSRF token.

Method + route	Authentication / role	Purpose and implementation note
GET, POST /account/preferences/secure	Authenticated	Demonstration token generation and constant-time validation.
GET /api/files/download	Public	Traversal can escape public subfolder only within lab_files root.
GET /api/files/download/secure	Public	Resolved path must remain inside public_files.
GET /health	Public	Simple health response for local operational confirmation.

What the paired examples prove and do not prove

A secure sibling helps explain one coding control in a deliberately small context. Parameter binding demonstrates query/data separation, HTML escaping addresses a response context, textContent avoids parsing a fragment as markup, a CSRF token binds a form submission to session state, a directory containment check blocks the fixture escape, and ownership checks bind an object to an identity. These examples are teaching contrasts, not complete hardened production implementations. The demo app also contains intentionally weak configuration and a hard-coded lab secret.

Framework implementation map by module

Module / file	Responsibility	Input -> output / bridge
framework/main.py	Run coordinator, target guard, stage recorder, overall status.	Target URL and optional ZAP report -> assessment ID and ordered stage records.
framework/config.py	Project name, target URL, DB URI, demo roles, role policies, API candidate paths.	Shared bounded configuration consumed by coordinator, scanner, dashboard, and verifier.
framework/dashboard.py	Loopback HTTP server, HTML assembly, static asset/report serving, run/retest forms, non-concurrency lock.	Browser POST -> workflow invocation -> redirect to assessment ID. Report route -> exact assessment artifact.
framework/recon/web_crawler.py	Breadth-first same-netloc HTML crawler, depth limit 2, GET only; extracts links and response status.	Base URL and optional authenticated session -> endpoint method/path/status/source.
framework/recon/api_discovery.py	Probes configured candidate paths and classifies status (accessible, restricted, redirect, etc.).	Role session plus path list -> API observations.
framework/recon/discovery.py, save_endpoints.py	Normalizes path schema and upserts Endpoint and EndpointAccess.	Method/path/source and role status -> durable access matrix inputs.
framework/roles/access_matrix.py	Builds method/path -> role -> status map from saved rows.	Target ID -> compact role access view.
framework/verification/authorization.py	Compares observed roles to policy; saves policy and response observations; checks object ownership.	Assessment, roles, paths, expected role lists -> candidate access and IDOR findings.
framework/verification/idor_verifier.py	Captures object status, content type, body length/hash, identity, owner, and classification.	Authenticated session and object path -> owned, cross-user candidate, or inconclusive observation.
framework/scanners/common_web.py	Local-only probes for SQL, reflected/stored/DOM XSS, CSRF surface, path traversal, headers/cookies/debug paths.	Assessment ID and local base URL -> finding + typed evidence record.
framework/scanners/zap_parser.py, zap_importer.py	Parses JSON alerts, validates report sites/ports/scheme, groups per plugin/name, imports evidence as candidates.	Scoped ZAP JSON -> normalized framework findings/evidence.
framework/scanners/zap_verifier.py	Safe local HTTP response checks for selected imported alerts.	ZAP report and target -> per-instance live verification JSON.
framework/verification/zap_triage.py	Attaches matching live checks as evidence to associated ZAP findings.	Assessment ID and verification JSON -> evidence, no status transition.
framework/verification/finding_manager.py	Reuse matching finding; serialize and deduplicate identical evidence payloads.	Assessment/title/category + typed evidence -> persistent IDs.

Module / file	Responsibility	Input -> output / bridge
framework/risk/scoring.py, assess_findings.py	Validates 1-5 factor inputs, weighted normalization, project thresholds, stores factors/method.	Candidate category -> score, preliminary label, factor JSON.
framework/attack_graph/engine.py	Allows selected evidence types to build individual paths; groups multiple role failures into compound authorization hypothesis.	Finding evidence -> path type, linked finding/evidence IDs, explanation and candidate status.
framework/verification/correlation.py	Finds related finding evidence and shared endpoints, writes assessment correlation JSON.	Persisted findings/evidence -> relationships for triage.
framework/verification/record_remediation_retest.py	Rechecks protected routes under identities and order ownership; validates original target; deduplicates repeated retest evidence.	Assessment ID -> PASS/FAIL/INCONCLUSIVE check evidence attached to original findings.
framework/verification/finding_lifecycle.py	Validates CANDIDATE -> VERIFIED -> REMEDIATED -> REOPENED transitions, requires reason/actor and suitable retest gates.	Explicit reviewer action -> state history with evidence links.
framework/reporting/generate_report.py	Collects DB rows and matching per-assessment JSON artifacts; writes JSON and HTML with summary counts.	Assessment ID -> reproducible report files.

Supporting modules that are not in the central run

The repository also includes an Nmap runner/parser and asset persistence modules, debug scripts, cleanup and migration utilities, and unit tests for crawler, API discovery, role checks, risk scoring, persistence, correlations, path lifecycle, and retest behavior. The current framework/main.py central pipeline does not call Nmap discovery or persist assets; the assessment #15 report accordingly has asset_count 0. Do not describe Nmap as an active stage of the button workflow. The existing test files are evidence of intended checks, not a claim that tests were run for this document.

How one assessment run executes in code

1. Request and scope validation

The dashboard binds ThreadingHTTPServer to 127.0.0.1:8010. A POST to /run-assessment is limited to a small form body, reads target_url, acquires a process-local lock, and calls run_assessment. run_assessment independently requires HTTP, localhost/127.0.0.1, the configured port, no credentials in the URL, and an empty or root path. The target scanner repeats local validation. This is defense in depth for the intended lab scope, not a network firewall.

2. Durable assessment and target identity

initialize_database creates tables. create_assessment writes an Assessment with CREATED status and a related Target, then main changes the assessment to RUNNING. Every later finding, endpoint, path, and stage points to that assessment or its target. IDs let the dashboard retrieve a run without confusing one run's evidence with another.

3. Stage isolation and failure behavior

run_stage catches an exception from each operation, records FAILED plus exception type/message, and returns false. The coordinator continues through independent stages where possible, marks unavailable ZAP stages SKIPPED, and sets overall PARTIAL if any required outcome is false. The report-generation stage itself is recorded; a final refresh includes that stage. Stage detail strings are capped at 4000 characters.

4. Role-aware discovery

The coordinator crawls once as anonymous and again after login for each configured role (ordinary user Alice and admin). It uses a depth-2 breadth-first crawler for same-netloc links, then probes an explicit API path list for each authenticated role. save_endpoints upserts a method/path pair for the assessment target and stores a separate status row per role. The list includes secure comparison routes so the report can show how target surfaces differ by identity.

5. Controlled checks

Authorization checks use GET and do not intentionally mutate the application. Common probes are targeted at known demonstration routes. The stored-XSS test is the exception that creates a review record; it uses a non-executable em formatting marker and attempts admin-authenticated deletion afterward. CSRF inspection only GETs forms. Path traversal requests only the repository fixture. No test sends a request to a non-local target.

6. Import, live verification, and triage

If a ZAP JSON file exists, the importer validates all report sites against configured host/port/scheme (with host.docker.internal also permitted for a Docker-origin report), parses alert instances, groups same plugin/name under one finding, and stores each affected URL as evidence. Scanner severity is kept separate from the framework risk score. Safe live response results are written to an assessment-specific JSON file and matching results are attached as evidence; finding state remains untouched.

7. Risk, paths, correlations, report

Risk is assigned only for known candidate vulnerability categories with configured policies. Attack-path builder recognizes selected evidence types and can build a compound role-boundary path only when at least two qualifying authorization findings exist. Correlation uses persisted evidence and endpoints. The report generator re-reads the database and only accepts correlation/live ZAP files whose embedded assessment ID matches. It writes the JSON first-class schema plus a human-readable HTML rendering.

Run lifecycle

CREATED -> RUNNING -> COMPLETED or PARTIAL. Individual stages store their own result regardless of the overall outcome. A skipped ZAP input stage contributes to a partial assessment in current orchestration; a run without the optional artifact should be described accurately as partial. Historical runs remain in the assessment DB.

Database model and what persists

Entity	Main fields	Why it exists
Assessment	id, target_url, status, created_at	One auditable execution and its overall state.
Target	id, assessment_id, url, status	Target context for one assessment; provides stable relation for endpoint records.
AssessmentStage	assessment_id, name, status, details, started_at, finished_at	Timing and outcome of each pipeline stage.
Asset	assessment_id, host, port, protocol, state, service, version, discovered_by	Schema for host/service inventory; not populated by central current pipeline.
Endpoint	target_id, method, path, discovered_by	Discovered application request surface.
EndpointAccess	endpoint_id, role, status_code, discovered_by	Observed response per identity role for access matrix.
Finding	assessment_id, title, vulnerability_type, severity, status, description, risk_score, risk_severity, risk_factors, risk_method	Issue summary plus lifecycle and project risk fields.
Evidence	finding_id, evidence_type, data, created_at	Typed structured observation attached to a finding. Data is JSON serialized in Text.
AttackPath	assessment_id, name, description, severity	Path candidate and linked IDs/evidence serialized in description JSON.
FindingStatusHistory	finding_id, previous_status, new_status, reason, actor, evidence_ids, transitioned_at	Audit trail for reviewer-driven lifecycle transitions.

Database behavior details

- Framework database uses SQLAlchemy and SQLite at framework/assessment.db by default. initialize_database creates missing tables; migration utilities support schema evolution.
- Target uses Flask-SQLAlchemy and SQLite at target-app/instance/security_lab.db, mounted into the container via Compose.
- Endpoint save logic upserts by target_id, method, and path; access rows upsert by endpoint and role.
- Finding save logic reuses by assessment_id, title, and vulnerability type; repeated identical evidence JSON is not inserted twice.
- Retest deduplication excludes the timestamp when comparing stable target/check content, so an identical retest does not produce a duplicate record.
- Per-assessment naming applies to report/correlation/live-check artifacts so reports do not accidentally mix runs.

Evidence examples

Evidence type	Key details retained
authorization_observation	role, path, status, content type, response length, redirect, SHA-256, small preview, classification
idor_ownership_observation	authenticated user ID, object owner ID, object ID, status, content type, hash, classification
controlled_sql_injection_probe	control/boolean/error statuses and product counts; whether difference/error disclosure observed; secure route name
controlled_reflected_xss_probe	whether the probe string reflected raw, secure comparison and escaping, content type; no browser execution
controlled_stored_xss_probe	temporary inert marker and escaped comparison outcome; note of cleanup/no script execution
controlled_csrf_form_observation	authenticated role, token field presence, secure route comparison, form not submitted
controlled_path_traversal_probe	fixture marker result and secure comparison response; note restricting read to lab fixture
security header/cookie observations	missing header names, server header, cookie flags; Secure attribute explicitly not assessed on HTTP lab
remediation_retest	target, time, exact checks, pass/fail/inconclusive values, all_checks_passed, status unchanged

Probe-by-probe mechanics and safety properties

SQL injection

The scanner requests /api/products/search with a no-match control term, a boolean expression, and a quote intended to elicit a database error. It compares HTTP status and product counts and checks for an exposed database_error in a 500 response. It records only a candidate when response behavior indicates a SQL-dependent difference or error disclosure. /api/products/search/secure uses a bound pattern as comparison. Probes are GET requests; result is not proof that arbitrary data extraction is possible.

Reflected XSS

The scanner requests /products/preview with an inert string containing markup-like syntax, checks whether the exact string appears unescaped, and checks the secure route for escaped output. It does not open a browser or run JavaScript. Review must still establish browser parsing context and actual impact.

Stored XSS and cleanup

The scanner posts a unique review that contains only an em element as a formatting test. It obtains the review ID, loads the vulnerable and secure review pages, and compares raw versus escaped markup. In a finally block it logs in as admin and calls the protected DELETE endpoint. The scanner notes that no script executed. Operational caveat: the cleanup attempt depends on admin login and the current route; its HTTP delete result is not separately asserted before returning. A reviewer should check the review list if a run is interrupted.

DOM source/sink heuristic

The scanner retrieves both DOM preview HTML responses. In the vulnerable response it checks that window.location.hash appears near .innerHTML (within 350 characters); in the secure response it checks textContent and absence of innerHTML. It does not supply a fragment payload, execute a browser, or establish runtime exploitability.

CSRF form inspection

The scanner authenticates as Alice and GETs vulnerable and secure preference pages. It confirms a form exists and whether an input named _csrf_token exists. It records that no state-changing request was submitted. This identifies missing token protection but does not independently prove cross-origin behavior or account for all SameSite/browser controls.

Path traversal fixture check

The scanner requests ../private/lab-note.txt via /api/files/download and checks for the fabricated TRAINING_LAB_PRIVATE_NOTE string. It compares the secure endpoint, which should reject the path. The vulnerable resolver still enforces a lab_files containment boundary, so this does not read arbitrary operating-system paths.

Security configuration review

A GET to / records presence/absence of Content-Security-Policy, X-Content-Type-Options, and X-Frame-Options, plus Server fingerprint. A session login records HttpOnly and SameSite flags; Secure is intentionally not assessed over the HTTP-only lab. A small list of debug/metadata paths is checked by status only and response bodies are not retained. These observations are contextual and low confidence by default.

Authorization and IDOR

Configured policies define admin-only endpoints. The verifier logs in using anonymous, user, and admin sessions, records status and response metadata/hash/preview, and marks an unexpected successful response as a candidate. The IDOR verifier compares authenticated identity with the owner field on JSON objects. Retest discovers Alice's own order and a different owner's order through admin data, then expects own-user access, foreign-user denial, and admin access. A malformed or incomplete target response remains inconclusive.

> **Safe does not mean complete: checks deliberately use narrow, known routes and bounded inputs. They are designed to make the workflow observable, not to exhaustively test arbitrary applications.**

Finding lifecycle, risk, paths, and correlation in depth

Candidate is a deliberate first-class status

record_finding writes status CANDIDATE by default. It reuses a finding when the assessment ID, title, and vulnerability type match. It attaches new typed evidence but skips exact duplicate serialized evidence. On reuse it refreshes description and only fills severity if the existing severity was UNASSESSED; it does not downgrade lifecycle state. This keeps repeated observations connected to one issue record.

Reviewer transitions are explicit

Transition	Gate
CANDIDATE -> VERIFIED	Allowed by the lifecycle helper with non-empty reason, actor, and optional linked evidence IDs.
VERIFIED -> REMEDIATED	Requires latest persisted remediation retest to be PASS, plus reason/actor.
REMEDIATED -> REOPENED	Requires latest persisted retest to be FAIL, plus reason/actor.
REOPENED -> VERIFIED	Allowed with reason/actor and optional linked evidence.

The helper also validates that linked evidence IDs belong to the same finding and appends a FindingStatusHistory row. The current dashboard exposes retest, but it does not provide a UI control for these lifecycle transitions; the helper exists for an explicit reviewer workflow outside the dashboard UI.

Risk is an explainable project heuristic

For each category with a policy, factor inputs range from 1 to 5. Each is normalized as ((value - 1) / 4) * 100. The normalized impact, exploitability, and confidence are weighted 0.40, 0.35, and 0.25, then rounded to one decimal. Project thresholds: 80+ CRITICAL, 60+ HIGH, 40+ MEDIUM, 20+ LOW, otherwise INFO. Policies are hard-coded category defaults, not environment-specific calibration. Confidence affects prioritization but does not replace verification status.

Attack paths are constrained by evidence type

The path engine only considers known evidence types for Broken Access Control, IDOR/BOLA, SQL injection, XSS, CSRF, and Path Traversal. An individual path retains the finding ID and evidence IDs, then stores route-specific nodes/explanation and a CANDIDATE path status. An IDOR path needs both owned-object access and candidate cross-user access. A compound authorization path requires at least two eligible policy findings and matching unexpected-role successes. A path with no support evidence is skipped.

Path lifecycle summary

The report derives a lifecycle summary from linked finding statuses: candidate findings remain candidate; all verified, all remediated, partially remediated, reopened, mixed, or unlinked states are represented. The report explicitly sets path_exploitability_confirmed to false. This stops a graph visualization from overstating what the underlying evidence means.

Correlation

Correlation reads finding and evidence records, normalizes target URLs/endpoints, recognizes scanner plugin IDs, and relates findings that share routes, artifacts, or evidence patterns. It writes a per-assessment JSON correlation artifact. It assists triage and report navigation; it does not reclassify findings or prove a chain.

What assessment #15 demonstrates

Metric	Observed value	Interpretation
Overall status	COMPLETED	All 10 recorded stages succeeded for this run.
Findings	21 total; 21 candidate	None is marked verified, remediated, or reopened.
Project risk distribution	3 high, 5 medium, 10 low, 3 unassessed	These are project scores/labels, not CVSS.
Attack paths	7 candidate paths	Hypotheses tied to findings/evidence; not verified end-to-end exploitation.
Endpoints	21	Saved target endpoint observations reported for the run.
Assets	0	Current central run does not execute Nmap asset discovery.

The dashboard and generated reports

Assessment desk interface

- A loopback-only Python ThreadingHTTPServer on 127.0.0.1:8010 serves the page, dashboard.css, shared tokens/base CSS, and assessment reports.
- The page has target URL input, Run assessment button, selected assessment hero/status, four summary stats, ordered workflow table, finding/remediation table, retest button, attack-path list, assessment history, and project risk note.
- Assessment history is newest-first, capped at 12 entries; if no assessment ID is selected, the newest is shown.
- A process-local lock prevents overlapping assessment or retest jobs. The lock is held while long-running work executes synchronously; the page does not show live per-stage progress until the request completes.
- A busy run returns a conflict message. Reports are served only for a numeric ID and html/json extension. CSS asset names are allowlisted. The dashboard has no authentication because it is intended to stay on loopback.

HTML and JSON report contents

Report area	Fields or contents
report_metadata	Title, assessment ID, generation time, framework name, explicit note that score is not CVSS.
assessment and stages	Target URL, status, created time; each stage name, status, details, start and finish times.
targets/assets/endpoints	Target records; host/service assets (empty for current central run); endpoint method/path/source.
access_matrix	Endpoint ID/method/path/source and observed status code keyed by role.
summary	Finding/candidate counts; risk and status distributions; path, endpoint, asset, and stage counts; verified/remediated/reopened counts.
findings	Category/title/description, scanner severity, project risk and factors/method, status, typed evidence, lifecycle history, latest retest aggregate.
attack_paths	Name, severity, path type/status, finding IDs, evidence IDs, explanation, details, linked lifecycle summary, exploitability-confirmed flag.
correlation and ZAP live checks	Only per-assessment matching artifacts are attached; otherwise the report states no matching artifact exists.
HTML presentation	Summary and reviewer-oriented tables from the same report data.

Why both outputs matter

A PDF or HTML page is convenient for review and faculty presentation. JSON preserves machine-readable keys, evidence arrays, assessment identity, and stage records for scripts, comparison, and later integrations. Assessment IDs and per-run filenames make an output traceable to its source execution. The PDF guides explain the workflow; they are not generated assessment reports.

The visual redesign

The Hallmark Editorial pass establishes the same visual language across both web surfaces: warm paper background, dark ink, controlled green accent, editorial type pairing, square corners, ruled sections, and reduced reliance on card grids. Store has a restrained featured product/corridor layout and catalog rows instead of emoji product art or repetitive tiles. Dashboard uses an evidence desk layout with dense but scannable tables. Existing route hooks, form fields, IDs, and handlers were retained. The review list no longer seeds a fake testimonial; it displays an empty state until a real review exists.

Achievement statement, implementation status, and honest boundaries

What the project has achieved

- Built a locally usable online shop backed by database models and real request handlers, replacing disconnected static training content as the main demo surface.
- Connected ordinary UI actions to routes the security workflow can discover and probe.
- Implemented vulnerable and secure comparison routes for SQL binding, HTML escaping, DOM text handling, CSRF tokens, object ownership, and file containment.
- Created a single run orchestrator with local target validation, isolated stage status, overall completion/partial logic, and persisted assessment IDs.
- Added role-aware discovery and policy checks, candidate IDOR observation, controlled common vulnerability checks, and optional ZAP data import plus safe response verification.
- Persisted endpoints, roles, findings, evidence, risk factors, paths, stage records, lifecycle history, and retest evidence in a structured schema.
- Generated paired HTML and JSON reports, a workflow dashboard, report history, and an assessment-specific artifact path.
- Added a retest path with PASS/FAIL/INCONCLUSIVE evidence and reviewer-governed state transition rules.
- Updated storefront and assessment desk to one coherent editorial visual system and documented the demo in a separate runbook.

Implemented versus intentionally out of scope

Implemented now	Not implemented or not claimed
Local Flask target, products/orders/reviews/users, sessions, admin, real store routes	Production ecommerce, real payment processing, external deployment, enterprise identity
Loopback guard in assessment controller and Docker loopback port binding	Network-level firewall or protection if a user changes deployment configuration
Crawler, candidate API paths, role/access observations	Nmap as an active central workflow stage; current asset count is zero
Known local HTTP probes and paired safe-route observations	General-purpose crawling/fuzzing of arbitrary apps or authenticated API schemas
ZAP JSON import and safe live checks	Launching/managing ZAP; freshness of the checked-in baseline is not guaranteed
Candidate findings, evidence, heuristic risk, path candidates, correlation, reports	Automatic finding verification, automatic remediation, confirmed exploit chains, CVSS calculation
Retest evidence and guarded lifecycle helper	Lifecycle status edit controls in dashboard UI; retest only covers configured authorization/ownership checks
Hallmark-style design and functional basic dashboard/store flows	Automated screenshot render tier in Hallmark lint; puppeteer-core is not installed in this workspace.

Known caveats teammates should be able to answer

- Why are there 21 findings but no verified findings? The framework records candidate observations and expects human confirmation.
- Why can assessment status be PARTIAL? An optional ZAP stage is treated as skipped when the input report is absent; any failed/skipped outcome keeps the run partial.
- Why are scores not CVSS? The code applies its own 3-factor weighting and category policy.
- Why is there no payment? Checkout writes fixture/demo orders only.
- Does pressing Add to bag immediately create an assessment finding? No. It updates client bag state. The Run assessment action later checks target endpoints and persists evidence.
- Can I run this against a public site? No. The code rejects non-local targets and the target is intentionally vulnerable.

Where teammates make changes

Change request	Likely files
Add or update a vulnerable route	target-app/app.py plus UI/template behavior if required; add a secure comparison and controlled fixture when appropriate.
Add a test to the central workflow	framework/scanners/common_web.py or framework/verification/; register it in framework/main.py and ensure typed evidence is persisted.
Change target scope or roles	framework/config.py and main.py guard; update tests/docs and keep loopback scope.
Change evidence schema	framework/database/models.py plus migration scripts, evidence writer, report serializer, and report HTML.
Change risk policy	framework/risk/assess_findings.py and scoring.py; describe method as project heuristic.
Change path logic	framework/attack_graph/engine.py; preserve candidate-only semantics and evidence links.
Change retest/lifecycle	framework/verification/record_remediation_retest.py and finding_lifecycle.py; retain old evidence and explicit transition gates.
Change dashboard or design	framework/dashboard.py, framework/templates/dashboard.html, framework/static/dashboard.css; target storefront styles live under target-app/static.

The strongest project demonstration

Choose one example and trace it all the way: a known route is shown in the store; the configured scope allows only local requests; a check records an observed status or response property; finding_manager attaches typed evidence to an assessment-specific candidate; risk policy stores factors; attack-path logic links only supported evidence; the report exposes the details; a developer fixes code; retest appends evidence; a human records the lifecycle decision. This is the project's central contribution: bridges between application behavior and a reviewable security process.

Teammate handoff checklist

- Can start/stop both services and find the current assessment ID.
- Can explain the target DB versus assessment DB.
- Can point to one vulnerable route and its secure comparison.
- Can describe how stage failures and skipped ZAP input affect status.
- Can locate JSON evidence for a dashboard row.
- Can explain candidate, verified, remediated, and reopened states.
- Can name at least three non-goals and keep the target local.